AI project · Topic 5 | Logic + Search | Advanced Web Ranking data




SEO Quest Planner

It reads your real keyword rankings, works out *which* keywords need help and *why*, then plans the cheapest to-do list that fixes them. It can explain every step.

1. Read

Pulls positions for one keyword group from the AWR API: you, your competitors, this update vs. the last one.

→

2. Reason

33 logic rules turn the numbers into facts like "dropped", "urgent" or "a competitor is beating us".

→

3. Plan

A* search tries action orders until it finds the cheapest plan that gets every chosen keyword to "done".

In one sentence: it's a tiny AI SEO consultant. It never guesses: every suggestion comes with a "because…" that you can trace all the way back to your ranking data.

🧠 How it thinks

The knowledge base is ordinary first-order logic, the same thing you'd write on a whiteboard.

1 Facts 📌

Things AWR tells us, written as small statements:

```
Ranking('trail running shoes', 14)
PreviousRanking('trail running shoes', 5)
CompetitorRanking(…, 'runfast.example', 6)
```

2 Rules 📜

"If this is true, then that is true." Capital letters are blanks that fit *any* keyword:

```
Dropped(K) :- Ranking(K,P),
  PreviousRanking(K,P0), P − P0 ≥ 5.
```

A few of the 33 rules, in plain English

📉 **Dropped:** the keyword fell 5 or more places since the previous update.

🎯 **Striking distance:** we rank #4–#20, close enough that a little work moves it up.

🥊 **Beaten by C:** competitor C ranks higher than us on this keyword.

🩹 **Needs work:** we're off page 1, *or* it dropped, *or* a competitor beats us while we're in striking distance.

🚨 **Priority 1 / 2 / 3:** drops first, quick wins second, everything else third.

🌱 **Healthy:** tracked, and *no* "needs work" rule applies. "Not" here means "nothing in the data proves it" (negation as failure).

🍰 Layers (strata)

A rule that says "*not* X" must wait until every X is known. So the engine works in layers: first the basic facts, then "dropped", then "needs work", then "healthy". That way an answer never flips later.

🔎 Ask it anything

The Query tab takes questions like `BeatenBy(K, C)` ("who beats us, and where?") and shows a **proof tree**: the rule used, the facts behind it, and the maths it did.

A plan, start to finish

Sample data, top 2 most urgent keywords. A* found 12 steps costing 22 effort points.

“trail running shoes”

Priority 1 · dropped

#1 · was #5 · runfast #6 · sportzone #11 · you #14 · #20

1	Audit the ranking page and the search results	1
2	Investigate why it dropped (indexing, cannibalisation, SERP changes)	2
3	Study the 2 competitors ranking above us	2
4	Add the topics competitors cover that our page misses	4
5	Add internal links pointing to the page	1
6	Schedule an AWR ranking update to measure the effect	1

“womens running shoes”

Priority 1 · #2 → #9

Same six-step quest. It is only allowed to start once the more urgent keyword's work is finished (a “blocked” rule enforces this).

Why step 4? The planner may only pick an action once its conditions are proven true:

- the competitors for “trail running shoes” have been analysed ✔ (step 3 did that)
- we have a page for it ✔ (we rank #14, so some page of ours exists)
- its content hasn't been improved yet ✔

Search race (3 keywords): how many plans each algorithm had to look at



BFS finds the plan with the fewest *steps*, which isn't the cheapest. UCS and A* both find the cheapest, but A*'s heuristic (a careful under-estimate of the work left) lets it skip about 94% of the search.

🧰 The toolbox

10 actions. Each has conditions (written as logic queries), effects, and an effort cost.

🔍	**Audit page**: look at the ranking page and the search results	🪙 1
🕵️	**Investigate drop**: only if it dropped	🪙 2
🥊	**Analyse competitors**: only if someone beats us	🪙 2
🛠️	**Optimise on-page**: titles, headings, meta	🪙 2
✍️	**Close content gap**: add what competitors cover	🪙 4
📝	**Rewrite content**	🪙 5
📰	**Publish supporting article**: improves content *and* adds links	🪙 7
🏗️	**Create landing page**: when we have no page at all	🪙 8
🔗	**Add internal links**	🪙 1
🗓️	**Schedule re-check**: a new AWR update to measure the result	🪙 1

🖥️ Using it

1. Paste your AWR API key (kept in memory only, never saved)
2. Pick a project, keyword group, search engine and ranking date
3. Pick your website and how many keywords to plan
4. Get quest cards, a timeline, or a checklist with 🎉 confetti

⚡ Fast and private

Downloads one keyword group, not the whole update, with 8 requests running in parallel. Past dates are cached on disk, so opening them again is instant. Runs locally on `127.0.0.1`. No extra libraries, just Python.

📖 Tiny glossary

Unification
Matching `Ranking(K, P)` with a real fact to fill in K and P.

STRIPS action
Conditions → add facts / remove facts.

Forward chaining
Apply rules again and again until nothing new comes out.

Admissible heuristic
An estimate that never over-promises, so A* stays optimal.